

AlphaRouter: Quantum Circuit Routing with Reinforcement Learning and Tree Search

Wei Tang[†]

AWS Quantum Technologies
New York, NY, USA
weittang@amazon.com

Yiheng Duan[†]

AWS Quantum Technologies
Seattle, WA, USA
yiheng@amazon.com

Yaroslav Kharkov

AWS Quantum Technologies
New York, NY, USA
ykharkov@amazon.com

Rasool Fakoor

Amazon Web Services
Santa Clara, CA, USA
fakoor@amazon.com

Eric Kessler

AWS Quantum Technologies
New York, NY, USA
erikessl@amazon.com

Yunong Shi

AWS Quantum Technologies
New York, NY, USA
shiyunon@amazon.com

Abstract—Quantum computers have the potential to outperform classical computers in important tasks such as optimization and number factoring. They are characterized by limited connectivity, which necessitates the routing of their computational bits, known as qubits, to specific locations during program execution to carry out quantum operations. Traditionally, the NP-hard optimization problem of minimizing the routing overhead has been addressed through sub-optimal rule-based routing techniques with inherent human biases embedded within the cost function design. This paper introduces a solution that integrates Monte Carlo Tree Search (MCTS) with Reinforcement Learning (RL). Our RL-based router, called AlphaRouter, outperforms the current state-of-the-art routing methods and generates quantum programs with up to 20% less routing overhead, thus significantly enhancing the overall efficiency and feasibility of quantum computing.

I. INTRODUCTION

Quantum computing [20], [41] is an emerging computational paradigm, which has a potential to transform many industries, including optimization [18], machine learning [47], quantum chemistry and material science [2], [3]. A quantum computer comprises quantum bits (qubits) that share quantum edges with each other. Quantum computers perform computation by applying quantum operations (gates) to qubits, akin to classical Boolean circuits. Such quantum gates operate on pairs of qubits, creating correlation in their states. This phenomenon, called entanglement, constitutes the underlying potential advantage of quantum computing.

However, quantum computers face a key challenge in applying quantum gates. Due to physical constraints, quantum gates can only be applied to adjacent pairs of qubits on a quantum computer. In general, quantum computers do not have all-to-all connectivity. As a result, transforming high-level abstract quantum programs (logical circuits) into hardware executable quantum circuits (physical circuits) that satisfy the connectivity constraints is necessary to run quantum programs.

Quantum routers hence become a necessary component to tackle the challenge. To accommodate the sparse qubit

connectivity in quantum computers, routers must move pairs of remote logical qubits to adjacent physical qubits, so that they can be executed on a quantum computer. To generate physical circuits that comply with specific topologies, routers utilize specialized quantum operations known as SWAP gates. These gates are applied to adjacent pairs of physical qubits to exchange the positions of the corresponding pair of logical qubits, effectively swapping their locations.

Unfortunately, SWAP insertion is very costly. Quantum operations are inherently non-perfect and error-prone [22]. SWAPs introduce significantly more error to quantum programs, destroys accurate quantum data and makes quantum program outputs wrong. As a result, optimizing routers to minimize their added SWAP overhead is crucial for advancing quantum computing efficiency [38].

Minimizing SWAP gates in quantum programs is crucial but challenging, as it parallels the token swapping problem [55], recognized as at least NP-hard [12]. Currently, there are two primary approaches to the circuit routing problem: satisfiability solver-based routing algorithms [33] and heuristics-based routing algorithms [1], [12], [24]. However, solver-based algorithms achieve optimal SWAP numbers but suffer from exponential time complexity with more qubits and quantum gates. On the other hand, heuristic routers generally produce a suboptimal solution with a high SWAP overhead. In contrast, machine learning methods for circuit routing problem allow to balance the solution quality and the runtime.

Instead, quantum circuit routing can be formulated as a sequential decision-making process by employing RL-based methods. Given the recent impressive results of Deep RL [4], [15]–[17], [19], [26], [30], [34], [37], [48], [49], adapting Deep RL frameworks to this problem offers several advantages. Firstly, an RL-based router can adapt to different benchmarks and quantum computer topologies through re-training or fine-tuning, eliminating the need for manual algorithm redesign. Secondly, RL agent models, particularly those utilizing transformers [54], are adept at handling longer input sequences. This

[†]Wei Tang and Yiheng Duan contributed equally to this work.

capability enhances their potential to achieve better optimization results, thereby significantly improving the efficiency of RL-based routing over traditional heuristic methods.

We developed an RL-based quantum circuit router, called AlphaRouter. AlphaRouter is inspired by AlphaZero [48], where it combines MCTS [11], [27] and RL [51]. MCTS excels in exploring a vast space of possible actions and outcomes, enhancing decision-making by balancing exploration and exploitation. Meanwhile, RL contributes by learning from the outcomes of these explorations, continuously improving strategy over time. This synergy allows for more sophisticated and globally optimized quantum compilation solutions. Our contributions hence include:

- 1) **SWAP reduction:** To the best of our knowledge, AlphaRouter is the first to integrate MCTS and RL to optimize for extended circuit sequences, in contrast against traditional methods that focus on single-layer local search. This enables AlphaRouter to reduce the number of SWAP gates by 10 – 20% over state-of-the-art routers.
- 2) **Unseen Benchmark:** AlphaRouter demonstrates robust performance and generalization ability across benchmarks, achieving consistent compilation results even for previously unseen benchmarks.
- 3) **Scalability** AlphaRouter exhibits scalable compilation efficiency, evidenced by a 15% reduction in the linear scaling coefficient for the number of SWAPs as benchmark sizes increase, while retaining a low inference time.

II. BACKGROUND

A. Quantum Circuit Routing

Figure 1a shows an example 5-qubit logical circuit visualized as a Directed Acyclic Graph (DAG). Logical quantum gates could apply to any pair of logical qubits. Each logical qubit goes through one or more quantum gates to evolve its quantum state. This is analogous to the classical computing process of applying operations to data stored in registers.

Figure 1b shows the topology and qubit mapping of a ring quantum computer with a trivial initial mapping, i.e. logical qubit l_i is mapped to physical qubit p_i , $\forall i \in \{0, \dots, 4\}$. Given the logical circuit and the quantum computer mapping, g_0 cannot be scheduled since the corresponding physical qubits of l_0 and $l_2 - p_0$ and p_2 – are not adjacent on the quantum computer. g_1 cannot be scheduled for the same reason. In addition, g_2 and g_3 must wait for g_1 to execute first because of data dependency. Instead, inserting a SWAP between p_1 and p_2 brings pairs of logical qubits (l_0, l_2) and (l_1, l_3) adjacent on the quantum computer. Figure 1c hence shows that both gates g_0 and g_1 can now be scheduled into the output physical circuit.

Figure 1d shows the remaining logical circuit after the first SWAP. Since the logical qubit targets of g_2 and g_3 are not adjacent on the quantum computer, they require an additional SWAP between p_3 and p_4 , as shown in Figure 1e. Consequently, Figure 1f shows that a second SWAP between p_3 and p_4 brings

logical qubits (l_1, l_4) adjacent on the quantum computer, thus enabling g_2 . With its dependence on g_2 fulfilled, g_3 is also scheduled, hence completing the routing process.

Routing strategically inserts SWAPs to reposition logical qubits on a quantum computer so that they become adjacent to execute the required logical gates. A SWAP could be applied to any physical edge on a quantum computer, exchanging the position of the two logical qubits. This process is guided by the quantum computer topology, the current qubit mapping and the remaining sequence of logical gates. Routing carries out a sequential decision making process to apply a SWAP gate and schedule all possible logical gates, given the current state. The routing process terminates once all the logical gates from the input circuit are successfully scheduled into the output physical circuit.

Existing quantum circuit routing methods [12], [32], [38] adopt local search that only considers the first few gates from the front layer of the circuit, leading to suboptimal routing quality.

B. Reinforcement Learning

In Reinforcement learning (RL), an agent learns to make decisions by interacting with an environment [51]. It involves the agent taking actions in the environment and receiving feedback in the form of rewards. Markov Decision Process (MDP) [43] is a mathematical framework commonly used to model and formalize RL problems. An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, T, r, \mu_0, \gamma)$, where $\mathcal{S}$ denotes the state space and $\mathcal{A}$ is the action space, the function $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_+$ encodes the transition probabilities of the MDP, μ_0 denotes the initial state distribution, $r(s, a)$ is the instantaneous reward obtained by taking action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$, and $\gamma \in [0, 1]$ is a discount factor for future rewards. The goal of RL is to find a policy, which is a mapping from states to actions $f : \mathcal{S} \rightarrow \mathcal{A}$, that maximizes the cumulative expected reward over time.

Figure 2 visualizes the standard RL framework. An agent, usually in the form of a neural network, interacts with an environment by taking actions a_t based on the current state s_t . Each action then leads to a new state s_{t+1} and produces a reward r_t , guiding the agent to learn optimal behaviors through trial and error to maximize cumulative rewards.

Given the sequential nature of quantum circuit routing, RL can be an effective tool for finding optimal routing. Section III details the formalization of quantum circuit routing, framing it as an MDP planning problem, and demonstrates how RL can efficiently and adaptively optimize the process.

C. Monte Carlo Tree Search

MCTS [11] is a search algorithm that progressively builds a search tree by exploring and evaluating potential moves, while focusing on exploring the high value regions. MCTS does not require supervision, but generates values for states iteratively.

Figure 3 shows the four stages in an MCTS cycle. An MCTS search tree consists of nodes corresponding to states s . In addition, each node stores statistics such as the number of visits $N(s)$ and the Monte Carlo value estimation $Q(s)$.

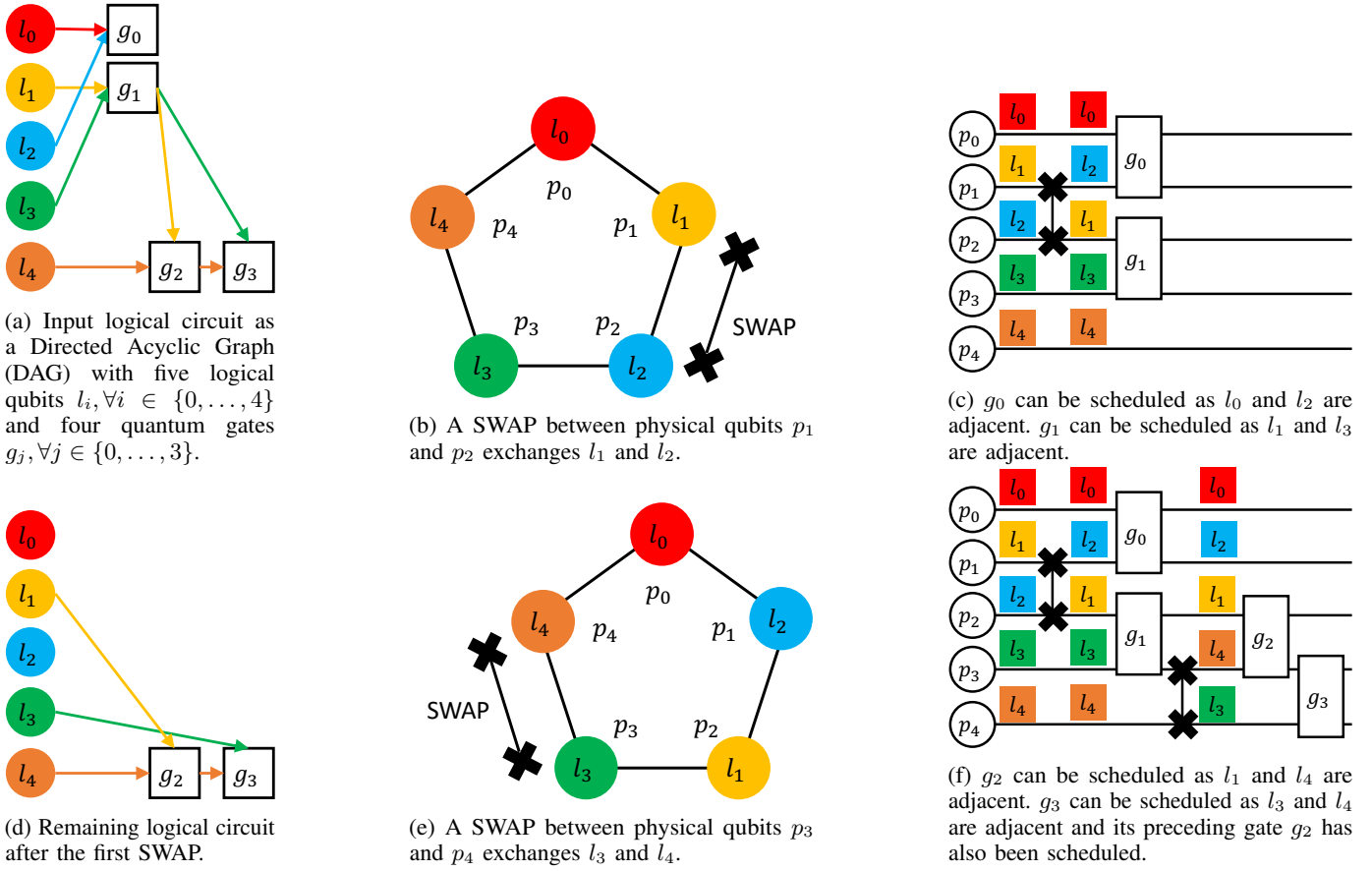


Fig. 1: Example of routing a logical circuit to a quantum computer using two SWAPs. Figures 1a, 1d show the logical circuits. Colors represent the flow of qubits through the gates. Figures 1b, 1e show the topology and qubit mapping of a ring quantum computer. $p_i, \forall i \in \{0, \dots, 4\}$ represent physical qubits. $l_i, \forall i \in \{0, \dots, 4\}$ represent the logical qubit mapped to a physical qubit. The black edges represent the connections. Figures 1c, 1f represent the routed physical circuit output. The gates are scheduled on their target logical qubits and topology compliant.

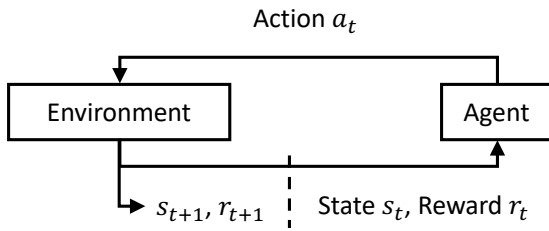


Fig. 2: A standard RL framework. An agent interacts with an environment to generate and learn from its experiences.

Throughout the iterative cycles of these stages, the values associated with each state in the tree are continuously updated.

The Selection stage greedily traverses the existing search tree from the root state till reaching a non-terminal child state. Greedy selections are based on maximising the Upper

Confidence Bound (UCB) [28] score in Equation 1.

$$UCB(s_t \rightarrow s_{t+1}) = \frac{Q(s_{t+1})}{N(s_{t+1})} + c \times \sqrt{\frac{\log N(s_t)}{N(s_{t+1})}} \quad (1)$$

where s_t indicates a parent node, s_{t+1} represents a child node to s_t , $Q(s)$ represents the current Monte Carlo value of a state s , $N(s)$ represents the number of visits to a state s , and c represents a hyperparameter tradeoff coefficient. UCB scores hence favor children states less visited and with higher Monte Carlo values.

The Expansion stage appends one or more new child states to the selected leaf state to expand the tree by applying each possible action. This represents exploring a new action from the leaf state.

The Simulation stage conducts a simulated roll-out from the leaf state by predicting the additional reward to a potential terminal state. Vanilla versions of MCTS usually follow a random policy to reach a terminal state. For example, when searching next moves for the game of Go, a vanilla MCTS implementation randomly plays stones to the current board

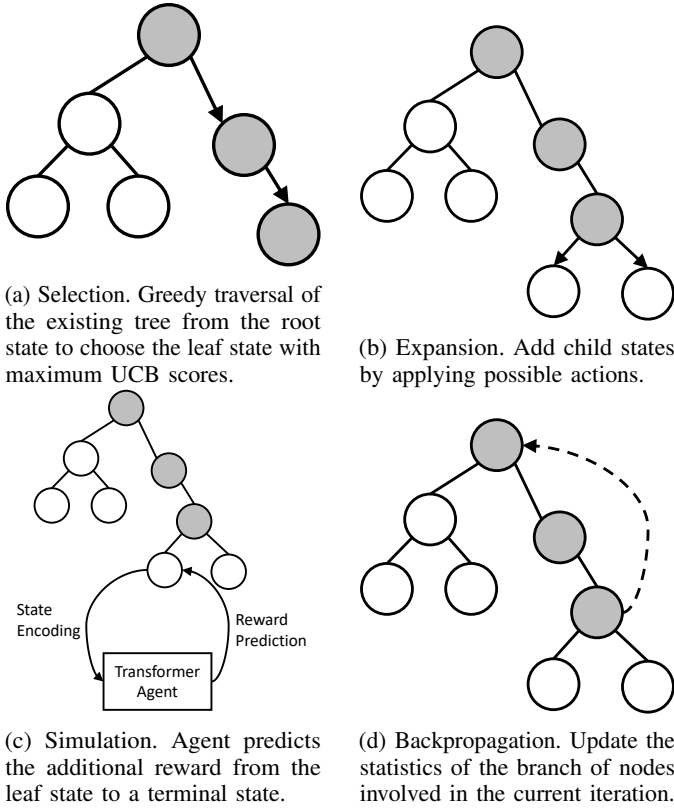


Fig. 3: MCTS with a transformer agent. The process repeats the four stages for a pre-determined max number of iterations. Circles in the tree represent states. Lines connecting the states represent actions.

until victory or loss. However, quantum circuit compilation is a decision process with an infinite horizon. This means that randomly inserting SWAP gates to the circuit does not guarantee compilation completion. Instead, AlphaRouter obtains a prediction of the value of the leaf state by feeding the leaf state encoding to a transformer-based agent, described in section IV-C.

The Backpropagation stage propagates the real rewards gathered from the simulation back to the root state. The statistics for each node within the selected branch, including metrics like value and visit count, are updated to reflect the results obtained from the simulation stage. This process ensures that the information at each node accurately represents the accumulated knowledge so far.

III. PROBLEM STATEMENT

Finding the minimum number of SWAPs to route a logical quantum circuit to a quantum computer can be modeled as an MDP. AlphaRouter focuses training a routing agent for a given quantum computer topology, i.e. with fixed number of qubits and connectivity. A quantum computer is modeled as a set of edges $E \equiv \{e : p_i, p_j\}$, where $P \equiv \{p_j\}$ represents the physical qubits. Similarly, $L \equiv \{l_j\}$ represents the logical qubits in an input quantum circuit.

The state space for circuit routing s_t comprises of the remaining logical circuit to be routed and the current qubit mapping. The state space is infinite as there are infinitely many logical circuits. A logical quantum circuit is expressed as logical gates $G_t \equiv \{g : l_{g,1}, l_{g,2}\}$, where $l_{g,1}$ and $l_{g,2}$ represent the two logical qubits of a gate g . The qubit mapping is a bijective mapping between physical and logical qubits $M_t : P \longleftrightarrow L$.

The action space is limited as there are $|E|$ possible SWAPs to insert, one for every quantum computer edge. A SWAP action A_t on edge $\{p_i, p_j\}$ exchanges the qubit mapping from $p_i, p_j \leftrightarrow l_i, l_j$ to $p_i, p_j \leftrightarrow l_j, l_i$ and updates the mapping M_t to M_{t+1} .

In addition, gates are sequentially checked for topology compliance and pending dependence on other gates. $g \in G_t$ satisfying the following conditions are scheduled and removed from G_t : (i) $l_{g,1}, l_{g,2}$ are adjacent on the quantum computer after the SWAP, i.e. $(M_{t+1}(l_{g,1}), M_{t+1}(l_{g,2})) \in E$ and (ii) $l_{g,1}, l_{g,2}$ have no other pending gates in G_t preceding g that had not been removed in the current time step.

The circuit routing problem is hence framed as: Route an input logical quantum circuit G_0 to a quantum computer backend E with an initial mapping M_0 . Find the shortest sequence of actions $\{s_t, A_t, s_{t+1}\}_{t=0}^{T-1}$ such that $|G_T| = 0$.

IV. FRAMEWORK

AlphaRouter combines both RL and MCTS to autonomously train a RL agent to predict the best placement of SWAPs given a logical quantum circuit and a quantum computer.

A. Training

Our training approach, inspired by AlphaZero, enables autonomous label generation and more efficient exploration for RL via MCTS. Figure 4 visualizes the overall training framework. The environment comprises both the remaining logical circuit to be routed and the current qubit mapping to produce a state observation s_t . Section IV-C details the state encoding. In addition, we limit the remaining logical circuit lookahead to be up to 48 logical gates. A transformer-based agent predicts the next SWAP action A_t by feeding the state s_t through its network as shown in Figure 6. A chosen SWAP action is then applied to advance the environment state.

A replay buffer stores the state experiences s_t and passes a randomly sampled experience batch $\{s\}$ to MCTS. MCTS undergoes its value update process in Section II-C for the sampled batch for 200 iterations (rollouts). Rewards r_t are defined to be the number of additional gates scheduled by applying action A_t , minus a penalty of incurring one extra SWAP:

$$r_t = |G_t| - |G_{t+1}| - 1 \quad (2)$$

Note that $r_t = -1$ if a SWAP fails to schedule any logical gates at time step t .

MCTS then outputs its action prediction for the sampled states $s' \in \{s\}$.

$$a_{MCTS}^* = \arg \max_a Q^{MCTS}(f(s', a)) \quad (3)$$

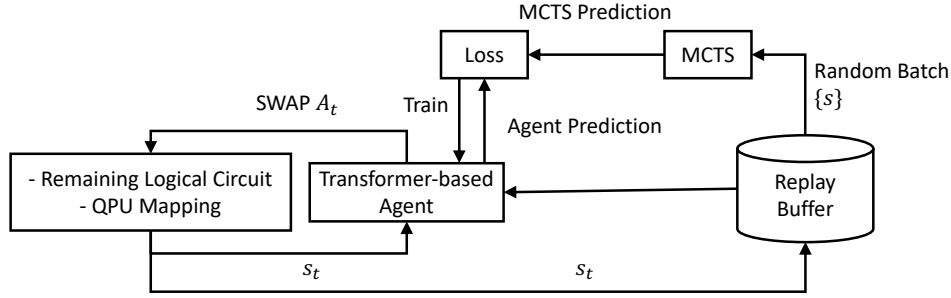


Fig. 4: AlphaRouter: MCTS + RL training framework.

where f is a state transition function $s_{t+1} = f(s_t, a)$, and $Q^{MCTS}(s)$ is the Monte Carlo value of a state s .

In addition, the agent outputs its action probability prediction by forwarding $s' \in \{s\}$ through its transformer network. The two state value predictions are then compared to compute a cross entropy loss:

$$l_1(s') = - \sum_a \delta(a_{MCTS}^*, a) \times \log(Q^\pi(s', a)) \quad (4)$$

where $\delta(a, a')$ is a delta function between two actions a and a' . $Q^\pi(s, a)$ is the transformer network action probability prediction for a given state s and action a .

The two state value predictions are also compared to compute a square loss:

$$l_2(s') = (Q^{MCTS}(s') - Q^\pi(s'))^2 \quad (5)$$

where $Q^\pi(s)$ is the transformer state value prediction for a state s .

Finally, an average loss combines both the cross entropy loss in Equation 4 and the square loss in Equation 5 to update the transformer network.

$$L = \frac{l_1 + \alpha l_2}{|E|} \quad (6)$$

where $|E|$ is the size of the action space. α is a hyperparameter tradeoff coefficient. α balances the two losses l_1 and l_2 . l_1 is the cross entropy loss between transformer network action probability prediction and MCTS action prediction. l_2 is the squared loss between MCTS and transformer SWAP count prediction for a given state. The tradeoff coefficient α is needed because larger benchmarks will require more SWAPs to finish compilation. As a result, l_2 naturally becomes larger and dominant for larger benchmarks. α is empirically determined based on the size of input benchmarks such that l_1 and l_2 are roughly of the same scale.

One of the key advantages of MCTS is its highly parallelizable nature, primarily due to the structure of its data and the relative independence of node expansions. This aspect of MCTS makes it well-suited for distribution across multiple compute nodes. In our implementation, we leverage this parallelizability by distributing the MCTS process across 16 *ml.c5.18xlarge* CPU instances on AWS SageMaker. Each

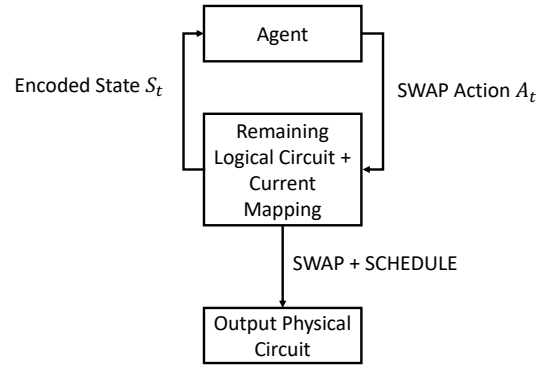


Fig. 5: AlphaRouter compiles quantum circuits with its trained agent via model inference without MCTS.

instance independently handles a portion of the tree search and node expansion, significantly enhancing the overall efficiency and speed of the search process. By dividing the workload in this manner, we can concurrently execute multiple tree searches and updates, accelerating the MCTS value update and making the most of the available computational resources.

B. Routing as Inference

After completing its training, the RL agent employs a standard iterative inference process to perform circuit routing. Figure 5 illustrates AlphaRouter's use of a trained agent for routing via model inference. The RL agent receives an encoded state s_t comprising the remaining logical circuit and current qubit mapping. Based on this, it then predicts and applies the next SWAP action a_t to adjust the qubit mapping. Any logical gates that now satisfy the topology are greedily scheduled. The iteration terminates when the input logical circuit has been fully routed.

In the standard AlphaZero [48] implementation, MCTS is utilized alongside the trained agent during the inference phase for additional refinement of predictions. However, this integrated approach tends to be slower compared to using the trained agent alone.

AlphaRouter, having demonstrated performance improvements over existing methods, enables us to omit the use of

MCTS during inference to achieve a much faster runtime. Our experiments show that excluding MCTS from the inference process significantly improves the runtime without compromising the routing performance. This enhancement in runtime is crucial for quantum computing, particularly because numerous quantum algorithms, such as variational quantum algorithms, necessitate the execution of thousands of distinct circuits to tackle benchmark problems. For context, the operational timeframe for executing a quantum circuit on a quantum processor is typically within the microsecond range. Consequently, the time taken for compilation can quickly emerge as a significant bottleneck in the standard workflow of quantum computing.

C. Agent Network

Figure 6 depicts the state encoding, beginning with the logical circuit DAG. In this DAG, each quantum gate is represented by its two qubits and a depth based on its topological order. For example, g_0 has physical qubit targets of $M(l_0)$ and $M(l_2)$ based on the mapping M . In addition, since g_0 does not depend on any other gates, it has a depth of 1. g_2 has direct dependency on g_1 and hence have depth of 2. Eventually, g_3 has depth of 3 as it depends on g_2 .

AlphaRouter observes the first few logical gates to limit the lookahead depth with a max number set at 48. Each gate vector then embeds its two physical qubit targets with a trainable lookup table. Each gate vector is subsequently represented as the mean of the two qubit embeddings, together with an extra dimension of depth. Overall, the node embedding process produces a feature matrix of shape Lookahead Length $\times$ (Embedding Dimension + 1).

Finally, this matrix is fed into a transformer-based actor-critic [29] network for encoding and decoding, yielding predictions including both SWAP action values and a state value. Figure 7 shows the transformer network architecture. AlphaRouter employs a 4-layer, 6-head transformer encoder architecture, starting with the embedded input shown in Figure 6.

The embedded input is first combined with positional encoding to retain sequence information. The Transformer architecture lacks inherent positional information in its structure because it processes input sequences as sets of elements without considering the order of these elements. To address this, Transformers often incorporate positional encodings [54] to provide the model with information about the position of each element in the sequence.

Positional encoding involves alternating sine and cosine functions that depend on the position of a quantum gate in the input sequence and the embedding dimension. This positional information is crucial for quantum circuit routing as the order of quantum gates carries physical meaning. Equations 7 shows the sinusoidal wave functions.

$$\begin{aligned} PE(pos, 2i) &= \sin(pos/10000^{2i/d}) \\ PE(pos, 2i+1) &= \cos(pos/10000^{2i/d}) \end{aligned} \quad (7)$$

Each layer comprises two main components: multi-head attention and a feed-forward network, both followed by an “Add & Norm” step for residual connections and layer normalization. The multi-head attention, with 6 attention heads, allows the model to focus on different parts of the input sequence simultaneously, enhancing its ability to capture complex dependencies. This is because self-attention enables each gate to interact with every other gate in the circuit, assessing their relationships and the overall sequence. Such an approach captures not only the individual qubit connections of each gate but also how gates influence one another across the circuit. This nuanced understanding of both local and global gate interactions is crucial for identifying optimal routing paths, as it mirrors the inherently interconnected nature of quantum computations, where the placement and order of gates significantly impact the best routing sequence. This structure repeats across all 4 layers, enabling the encoder to process and transform the input into a high-level, contextually enriched representation. A final linear layer serves as the decoder and outputs the action and state value predictions.

Table I summarizes AlphaRouter’s Tranformer model architecture and hyperparameters.

Number of attention layers	4
Number of attention heads	6
Node embedding dimension	20
Total number of parameters	2.2M
Optimizer	Adam
Learning rate	0.1
Learning rate decay	0.8
UCT coefficient c	$\sqrt{2}$
Batch Size	32

TABLE I: Transformer model parameters and training configuration.

D. Overall Algorithm

Algorithm 1 shows the overall training algorithm. The algorithm box uses the same notations as Section III. AlphaRouter uses the Adam optimizer [25] to train its transformer network.

V. EXPERIMENT SETTINGS

A. Quantum Computer Backends

We train and test the performance of AlphaRouter on various quantum computers include (i) *Tokyo*, an IBM quantum computer with 12 qubits arranged in a 3×4 grid; (ii) *OQC*, an Oxford Quantum Circuits quantum computer with 8 qubits arranged in a circle; and (iii) *Guadalupe*, an IBM quantum computer with 16 qubits in a heavy hexagon topology [9].

The training was performed using AWS SageMaker cloud service. The training runs for 100 episodes on *Random* benchmarks for all quantum computer topologies.

B. Initial Mapping

We employ two distinct strategies for initial qubit mapping: (i) trivial and (ii) random. The trivial initial mapping simply assigns the logical qubits in a numerical order to the physical qubits, i.e. $p_i = l_i$. On the other hand, random initial

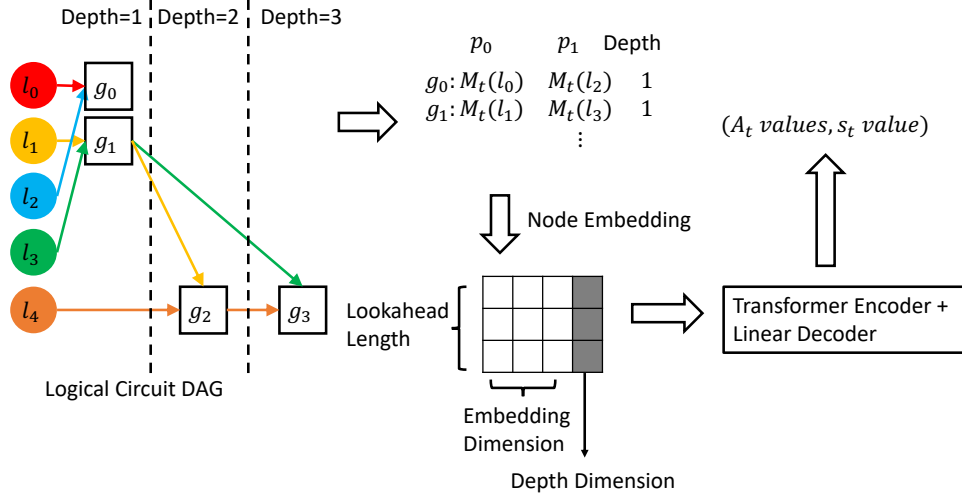


Fig. 6: Encoding the logical circuit G_t and the current qubit mapping M_t .

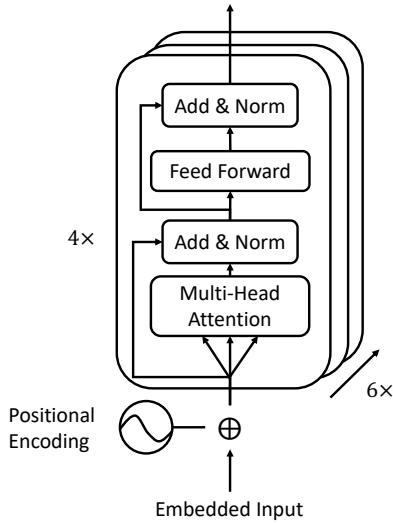


Fig. 7: AlphaRouter uses a transformer-based neural network as its agent. The model comprises 4 multi-head attention layers, each layer has 6 attention heads.

mapping introduces variability by randomly assigning qubits. AlphaRouter is designed to solve only the circuit routing problem for a given initial mapping. In the future it is viable to extend our approach to simultaneously solve both routing and initial mapping problems using the same RL model. One possible approach is to consider the initial mapping as a sequence of virtual SWAPs in the beginning of the routing.

C. Benchmarks

The benchmarks listed below are among the most commonly used quantum circuit types. We sample random circuit instances from each circuit class and compute the average SWAP count.

- 1) *Regular*: Quantum Approximate Optimization Algorithm (QAOA) solves the maximum independent set problem for random 3-regular graphs [46].
- 2) *Erdos*: The same algorithm as *Regular* but for random Erdos-Renyi graphs.
- 3) *QFT*: Quantum Fourier Transform [10] is a common subroutine in many quantum algorithms that promise speedup over classical algorithms, including the Shor's factoring algorithm.
- 4) *QV*: Quantum Volume circuits [13] quantify the largest random circuit of equal width and depth that a quantum computer successfully implements.
- 5) *GHZ*: Generates the entangled Greenberger-Horne-Zeilinger state [23].
- 6) *BV*: Bernstein Vazirani algorithm solves the hidden string problem more efficiently than classical algorithms do [6].
- 7) *HS*: Hidden Shift algorithm [53].
- 8) *Random*: Random quantum circuits consisting of uniformly sampled single and two-qubit gates.

Our benchmarks encompass a diverse array of circuit structures, ensuring a thorough and extensive assessment of our RL-based quantum circuit router's performance across various circuit complexities.

D. Baseline Routers

We use various Qiskit routers as the baseline comparisons. These are some of the most commonly used heuristics routers. Specifically, *Basic* is a greedy strategy that finds the shortest path to implement quantum gates sequentially. *Stochastic* is a randomized algorithm that performs multiple trials for SWAP insertion in each circuit layer. *SABRE* [32] is the state-of-the-art heuristics router. It performs a bi-directional search and uses a custom objective function, that takes into account SWAPs both in the front layer and the last layer in the circuit.

Algorithm 1: MCTS+RL Training

Input: Number of circuits N . Logical quantum circuits $\{G_{0,i}\}$. Initial qubit mapping $\{M_{0,i}\}$. Quantum computer topology E . Feature extraction function $\mathcal{O}$. State transition function f . Transformer network π .
Initialize $unfinishedIndices = \{1, \dots, N\}$.
Initialize $t = 0$.
Initialize replay buffer $B = empty$.
for each episode **do**
 for $i \in unfinishedIndices$ **do**
 State $s_{t,i} = \mathcal{O}(G_{t,i}, M_{t,i})$.
 Sample $A_{t,i}$ based on $Q^\pi(s_{t,i}, a)$.
 $s_{t+1,i} = f(s_{t,i}, A_{t,i})$.
 Add s_t to B .
 if s is a terminal state **then**
 remove i from $unfinishedIndices$.
 end if
 end for
if $|B| > 320$ **then**
 Random sample experience batch $\{s\}$.
 for Each state $s' \in \{s\}$ **do**
 Run MCTS for 200 rollouts, outputs the MCTS action prediction
 $a_{MCTS}^* = \arg \max_a Q^{MCTS}(f(s', a))$.
 Compute the action value predictions by the transformer agent $Q^\pi(s', a)$.
 Compute cross entropy loss according to Equation 4.
 Compute squared error according to Equation 5.
 end for
 Compute the average batch loss according to Equation 6.
 Update the transformer parameter with the Adam optimizer.
 end if
end for

VI. RESULTS

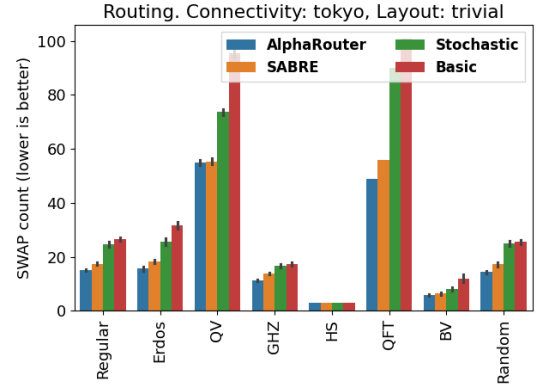
We designed 4 experiments to test a specific aspect of the RL compiler, while keeping the other factors constant.

A. Generalizing to Unseen Benchmarks

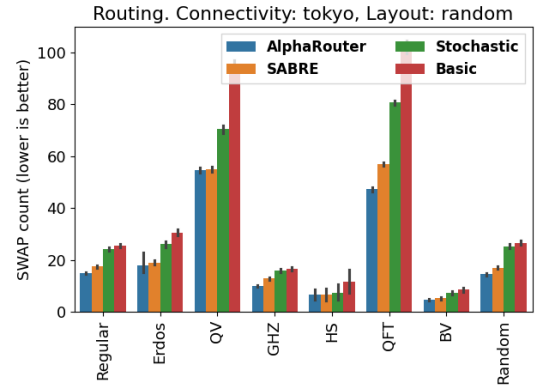
AlphaRouter was trained on the *Regular* benchmark and tested on a variety of benchmarks listed in Section V-C. Figure 8 compares the total number of SWAPs by AlphaRouter and Qiskit baseline routers. On average, the RL agent outperforms the baseline compilers for the majority of the benchmarks. Specifically, RL reduces the SWAP counts by 10 – 20% over the best baseline compiler in SABRE even for previously unseen benchmarks.

B. Scaling to Larger Benchmarks

Given a fixed benchmark type and a quantum computer topology, the number of SWAPs is expected to increase linearly with the length of the benchmark. However, this linear growth



(a) Trivial mapping. The error bars show the standard deviation from 20 circuits for each benchmark type. AlphaRouter reduces the number of SWAPs by $\sim 10 - 15\%$ compared to the state-of-the-art SABRE router.



(b) The same random initial mapping applies to both AlphaRouter and baseline routers. The error bars show the standard deviation from 5 random initial mappings.

Fig. 8: Average number of SWAPs by training AlphaRouter on *Regular* and testing it on various benchmarks. AlphaRouter outperforms baseline routers for the majority of the benchmarks.

varies across different routers, indicating their routing efficiency. We route *Random* benchmarks with an increasing number of gates on the Tokyo quantum computer using random initial mapping. Figure 9 shows that AlphaRouter has better scaling (lower linear coefficient α calculated by linear regression), compared to the baseline Qiskit routers by approximately 15%.

C. Adapting to Different Quantum Computers

Quantum circuit routers must be able to adapt to different quantum computers. We train AlphaRouter on IBM Tokyo (12q), IBM Guadalupe (16q), and OQC (8q) quantum computers with *Random* benchmark and test its performance on all benchmarks. Figure 10 shows the ratio of SWAPs of AlphaRouter versus SABRE. AlphaRouter excels on Tokyo and Guadalupe quantum computers, achieving 10 – 20% SWAP reduction for most benchmarks. Its performance comes close to SABRE on the OQC quantum computer. This is because

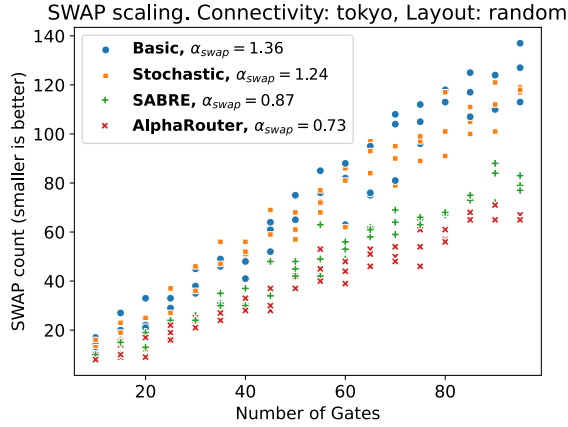


Fig. 9: Routing *Random* benchmark to the Tokyo quantum computer and random initial mapping for increasing number of gates. AlphaRouter demonstrates $\sim 15\%$ smaller scaling coefficient α_{swap} compared to SABRE.

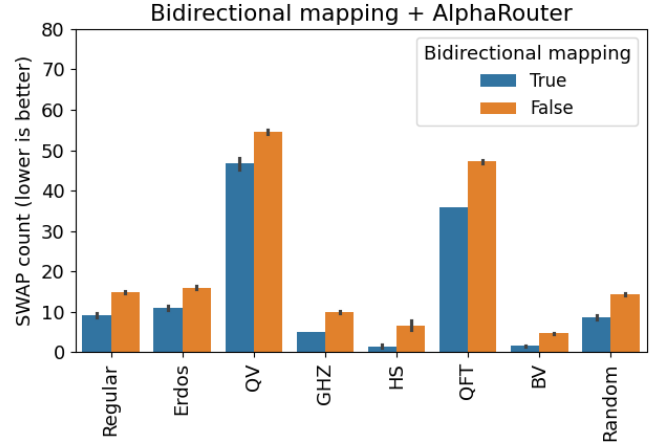


Fig. 11: AlphaRouter with and without bidirectional mapping to improve its initial mapping. Integrating the technique reduces 40% SWAPs on average on Tokyo quantum computer.

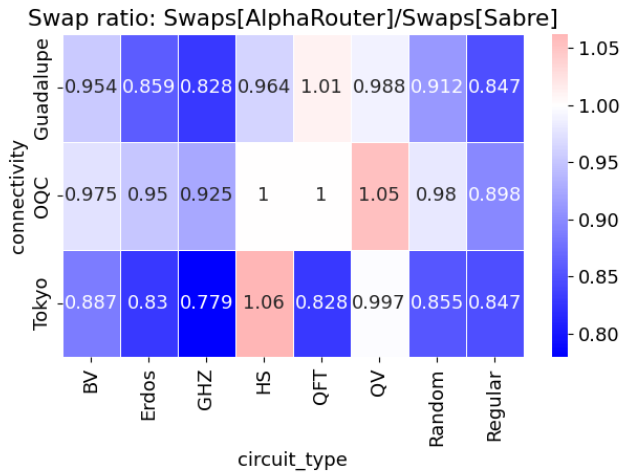


Fig. 10: Average swap ratios of AlphaRouter versus SABRE across different benchmarks and with random initial mapping. Lower is better. AlphaRouter adapts to different quantum computers, outperforming or matching SABRE.

OQC is a smaller quantum computer and has less room for improvement by RL.

D. Improving Initial Mapping

The SABRE router utilizes a bidirectional mapping technique to optimize initial qubit mapping. It starts with a forward pass to establish an initial mapping of logical to physical qubits, followed by a backward pass that refines this mapping using the complete circuit layout [32].

While AlphaRouter only solves SWAPs, it could integrate the bidirectional mapping technique to improve its routing overhead. Experiments in Figure 11 compare the AlphaRouter

SWAP counts with and without the bidirectional mapping pass. Notably, incorporating bidirectional initial mapping reduces 40% SWAPs on average across all benchmarks for the Tokyo quantum computer.

VII. RELATED WORK

Graph routing problems are well known problem in computer science and operations research [31]. Graph routing problems can be solved by using provable algorithms with exponential runtime in the worst case, heuristics based on mixed integer programming, or by employing a combination of planning and deep reinforcement learning methods [5], [7], [14], [35], [40], [45]. Routing problems are inherently classical and are not specific to quantum computing.

It is widely recognized that even state-of-the-art quantum routers suffer from a significant optimality gap in SWAP count [52]. [42] have implemented a Q-learning based RL approach for quantum circuit routing, utilizing a standard setup comprising an environment and an agent. [50] adopted a structure combining MCTS and RL, but their optimization focuses solely on a single layer of gates. In contrast, [57] employed MCTS without integrating RL, offering a different perspective on quantum circuit optimization. It is important to note, that [42], [50] focused on optimization of the circuit depth which defined the RL reward function, rather than optimizing the SWAP count overhead. Minimizing SWAP count is the most important metric for the efficient utilization of quantum computers rather than the gate depth. Each of these works contributes uniquely to the evolving landscape of quantum circuit compilation methodologies. In addition, several heuristic-based compilers [12], [32], [39] perform local optimizations but lack the scalability to perform global searches.

In addition, quantum compilation also involves other directions beyond improving routing. One direction of research

designs RL-based gate optimization algorithms, based on local mathematical identities between gates and local rewrite rules to simplify the input circuits before routing [21], [44]. In addition, RL has applied to continuous time control of quantum hardware, improving the accuracy of quantum operations [8], [36], [56]. These research cover different stages of the quantum compilation stack. This paper addresses the ubiquitous problem of circuit routing, which is necessary for quantum computers with a limited connectivity.

VIII. CONCLUSION

In this paper, we incorporate RL and MCTS to develop a quantum circuit router, distinctively free from arbitrary heuristic routing rules. Our approach not only exhibited 10 – 20% SWAP reduction over existing routers but also demonstrated remarkable abilities in benchmark generalization and scalability. Our pioneering work in integrating MCTS with RL for quantum circuit routing highlights the transformative potential of combining RL and MCTS for complex optimization tasks, advancing the development and practicality of quantum computing.

IX. ACKNOWLEDGEMENTS

We would like to thank Lihong Li for the insightful discussions throughout the project.

REFERENCES

- [1] MD Sajid Anis, Héctor Abraham, Rochisha Agarwal AduOffei, Gabriele Agliardi, Merav Aharoni, I Akhalwaya, Gadi Aleksandrowicz, Thomas Alexander, M Amy, S Anagolum, et al. Qiskit: an open-source framework for quantum computing (2021). DOI: <https://doi.org/10.5281/zenodo.2573505>, 2021.
- [2] Alán Aspuru-Guzik, Anthony D. Dutoi, Peter J. Love, and Martin Head-Gordon. Simulated quantum computation of molecular energies. *Science*, 309:1704 – 1707, 2005.
- [3] Bela Bauer, Sergey Bravyi, Mario Motta, and Garnet Kin-Lic Chan. Quantum algorithms for quantum chemistry and quantum materials science. *Chemical reviews*, 2020.
- [4] Marc G Bellemare, Salvatore Candido, Pablo Samuel Castro, Jun Gong, Marlos C Machado, Subhodeep Moitra, Sameera S Ponda, and Ziyu Wang. Autonomous navigation of stratospheric balloons using reinforcement learning. *Nature*, 588(7836):77–82, 2020.
- [5] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d’horizon. *ArXiv*, abs/1811.06128, 2018.
- [6] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473, 1997.
- [7] Aigerim Bogrybayeva, Meraryslan Meraliyev, Taukekhan Mustakhov, and Bissenbay Dauletbayev. Learning to solve vehicle routing problems: A survey. *ArXiv*, abs/2205.02453, 2022.
- [8] Marin Bukov, Alexandre G. R. Day, Dries Sels, Phillip Weinberg, Anatoli Polkovnikov, and Pankaj Mehta. Reinforcement learning in different phases of quantum control. *Physical Review X*, 2017.
- [9] Christopher Chamberland, Guanyu Zhu, Theodore J Yoder, Jared B Hertzberg, and Andrew W Cross. Topological and subsystem codes on low-degree graphs with flag qubits. *Physical Review X*, 10(1):011022, 2020.
- [10] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [11] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *Computers and Games*, 2006.
- [12] Alexander Cowtan, Silas Dilkes, Ross Duncan, Alexandre Krajenbrink, Will Simmons, and Seyon Sivarajah. On the qubit routing problem. *arXiv preprint arXiv:1902.08091*, 2019.
- [13] Andrew W Cross, Lev S Bishop, Sarah Sheldon, Paul D Nation, and Jay M Gambetta. Validating quantum computers using randomized model circuits. *Physical Review A*, 100(3):032328, 2019.
- [14] Paulo da Costa, Jason Rhuggenaath, Yingqian Zhang, Alp Eren Akçay, and Uzay Kaymak. Learning 2-opt heuristics for routing problems via deep reinforcement learning. *SN Computer Science*, 2, 2021.
- [15] Rasool Fakoor, Pratik Chaudhari, and Alexander J. Smola. P3O: policy-on policy-off policy optimization. In *Proceedings of the Thirty-Fifth Conference on Uncertainty in Artificial Intelligence, UAI 2019*, page 371, 2019.
- [16] Rasool Fakoor, Pratik Chaudhari, Stefano Soatto, and Alexander J. Smola. Meta-q-learning. In *International Conference on Learning Representations*, 2020.
- [17] Rasool Fakoor, Jonas W Mueller, Kavosh Asadi, Pratik Chaudhari, and Alexander J Smola. Continuous doubly constrained batch reinforcement learning. *Advances in Neural Information Processing Systems*, 34:11260–11273, 2021.
- [18] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv: Quantum Physics*, 2014.
- [19] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 10 2022.
- [20] Richard P Feynman. Simulating physics with computers. *Int. j. Theor. phys.*, 21, 1982.
- [21] Thomas Fösel, Murphy Yuezhen Niu, Florian Marquardt, and Li Li. Quantum circuit optimization with deep reinforcement learning. *arXiv preprint arXiv:2103.07585*, 2021.
- [22] Diana Franklin and Frederic T. Chong. Challenges in reliable quantum computing. 2004.
- [23] Daniel M Greenberger, Michael A Horne, and Anton Zeilinger. Going beyond bell’s theorem. In *Bell’s theorem, quantum theory and conceptions of the universe*, pages 69–72. Springer, 1989.
- [24] Yaroslav A. Kharkov, Alexandra Ivanova, Eugene A. Mikhantiev, and Aleksandr I. Kotelnikov. Arline benchmarks: Automated benchmarking platform for quantum compilers. *ArXiv*, abs/2202.14025, 2022.
- [25] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [26] B Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A Al Sallab, Senthil Yogamani, and Patrick Pérez. Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(6):4909–4926, 2021.
- [27] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, 2006.
- [28] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.
- [29] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.
- [30] Nevena Lazic, Craig Boutilier, Tyler Lu, Eehern Wong, Binz Roy, MK Ryu, and Greg Imwalle. Data center cooling using model-predictive control. *Advances in Neural Information Processing Systems*, 31, 2018.
- [31] Jan Karel Lenstra and Alexander H. G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11:221–227, 1981.
- [32] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for nisq-era quantum devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1001–1014, 2019.
- [33] Wan-Hsuan Lin, Jason Kimko, Bochen Tan, Nikolaj S. Bjørner, and Jason Cong. Scalable optimal layout synthesis for nisq quantum processors. *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2023.
- [34] Yao Liu, Pratik Chaudhari, and Rasool Fakoor. Budgeting counterfactual for offline rl. In *Advances in Neural Information Processing Systems*, volume 36, pages 5729–5751, 2023.
- [35] Xu Mai, Quanzhi Fu, and Yi Chen. Packet routing with graph attention multi-agent reinforcement learning. *2021 IEEE Global Communications Conference (GLOBECOM)*, pages 1–6, 2021.
- [36] Friederike Metz and Marin Bukov. Self-correcting quantum many-body control using reinforcement learning with tensor networks. *Nature Machine Intelligence*, 5:780 – 791, 2022.

- [37] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv:1312.5602*, 2013.
- [38] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '19, page 1015–1029, New York, NY, USA, 2019. Association for Computing Machinery.
- [39] Giacomo Nannicini, Lev S Bishop, Oktay Günlük, and Petar Jurcevic. Optimal qubit assignment and routing via integer programming. *ACM Transactions on Quantum Computing*, 4(1):1–31, 2022.
- [40] M. Nazari, Afshin Oroojlooy, Lawrence V. Snyder, and Martin Takác. Reinforcement learning for solving the vehicle routing problem. In *Neural Information Processing Systems*, 2018.
- [41] Michael A Nielsen and Isaac L Chuang. Quantum computation and quantum information. *Phys. Today*, 54(2):60, 2001.
- [42] Matteo G Pozzi, Steven J Herbert, Akash Sengupta, and Robert D Mullins. Using reinforcement learning to perform qubit routing in quantum compilers. *ACM Transactions on Quantum Computing*, 3(2):1–25, 2022.
- [43] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. USA, 1st edition, 1994.
- [44] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. Compiler optimization for quantum computing using reinforcement learning. *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2022.
- [45] Joao Reis, Miguel Rocha, Truong Khoa Phan, David Griffin, Franck Le, and Miguel Rio. Deep neural networks for network routing. In *2019 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2019.
- [46] Zain H Saleem, Teague Tomesh, Bilal Tariq, and Martin Suchara. Approaches to constrained quantum approximate optimization. *arXiv preprint arXiv:2010.06660*, 2020.
- [47] Maria Schuld and Francesco Petruccione. *Quantum Models as Kernel Methods*, pages 217–245. Springer International Publishing, Cham, 2021.
- [48] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- [49] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [50] Animesh Sinha, Utkarsh Azad, and Harjinder Singh. Qubit routing using graph neural network aided monte carlo tree search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9935–9943, 2022.
- [51] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [52] Bochen Tan and Jason Cong. Optimality study of existing quantum computing layout synthesis tools. *IEEE Transactions on Computers*, 70:1363–1373, 2020.
- [53] Wim Van Dam, Sean Hallgren, and Lawrence Ip. Quantum algorithms for some hidden shift problems. *SIAM Journal on Computing*, 36(3):763–778, 2006.
- [54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [55] Katsuhisa Yamanaka, Erik D Demaine, Takehiro Ito, Jun Kawahara, Masashi Kiyomi, Yoshio Okamoto, Toshiki Saitoh, Akira Suzuki, Kei Uchizawa, and Takeaki Uno. Swapping labeled tokens on graphs. *Theoretical Computer Science*, 586:81–94, 2015.
- [56] Jiahao Yao, Paul Körtter, Hans Gundlach, Lin Lin, and Marin Bukov. Noise-robust end-to-end quantum control using deep autoregressive policy networks. In *Mathematical and Scientific Machine Learning*, 2020.
- [57] Xiangzhen Zhou, Yuan Feng, and Sanjiang Li. Quantum circuit transformation: A monte carlo tree search framework. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 27(6):1–27, 2022.